

Artificial Intelligence

Assignment 01

Course Title: Artificial Intelligence (AI2002)

Total Marks: 150 Marks | **Work Mode:** 2-3 people

Programming Language: Python 3

Deadline: 27th September 2026

1. Instructions

Step 1: Download & Workspace Setup

- Download search.zip folder
- Extract the archive into your working project directory. Ensure the relative folder structure (like layouts/) remains intact.
- Verify that Python 3 (Python 3.7+) is configured on your machine by running `python --version` or `python3 --version` in your terminal.
- Test running the base environment without search logic:
`python pacman.py`

Step 2: Files You Are Permitted to Edit / Create

- `search.py`: **PRIMARY EDITABLE FILE**. All core search algorithms (DFS, BFS, UCS, GBFS, A*) and automated CSV logging logic must be written here.
- `searchAgents.py`: **SECONDARY EDITABLE FILE**. All state space formulations, custom heuristic functions, and agent wrappers (e.g., `CornersProblem`, `FoodSearchProblem`) must be implemented here.
- `layouts/Search.lay`: **CUSTOM FILE TO ADD**. Create one original layout file inside the `layouts/` folder named.

Step 3: Strictly Forbidden File Modifications

To ensure full compatibility with autograders(for testing not for grading) and live evaluation scripts, **DO NOT EDIT** any of the following core system files:

- `pacman.py`: The main game loop runner, game state controller, and CLI argument parser.
- `game.py`: Underlying physics, grid data structures, directions (North, South, East, West), agent actions, and state representations.
- `util.py`: Data structures including `Stack`, `Queue`, and `PriorityQueue`. Use these provided classes exclusively.
- `layout.py`, `graphicsDisplay.py`, `graphicsUtils.py`, `textDisplay.py`: Maze layout loader and graphic rendering components.

Welcome to the Pacman Search Game Project!

*All those colored walls,
mazes give pacman the blues,
so teach him to search.*



2. Tasks

Task 1: Depth-First Search (DFS)

Goal: Implement graph-search Depth-First Search in the function `depthFirstSearch` inside `search.py`.

Technical Requirements:

- Use a LIFO Stack from `util.py` as your fringe/frontier.
- Implement strict graph search: maintain an explicit explored set to prevent visiting previously expanded states and avoiding infinite loops.
- Return a valid list of actions (e.g., ['North', 'East', 'South']) that leads Pacman from start state to goal state.
- Follow the mandatory successor expansion order: **North** → **East** → **South** → **West**.

```
python pacman.py -l tinyMaze -p SearchAgent -a fn=dfs
python pacman.py -l mediumMaze -p SearchAgent -a fn=dfs
python pacman.py -l bigMaze -z .5 -p SearchAgent -a fn=dfs
```

Task 2: Breadth-First Search (BFS)

Goal: Implement graph-search Breadth-First Search in `breadthFirstSearch` inside `search.py`.

Technical Requirements:

- Use a FIFO Queue from `util.py` as your fringe/frontier.

- Ensure that for unweighted graphs (where every step cost is equal to 1), BFS guarantees finding the shallowest/optimal path in terms of total action steps.
- Avoid re-enqueuing states that are already present in the frontier or explored set.

```
python pacman.py -l mediumMaze -p SearchAgent -a fn=bfs
python pacman.py -l bigMaze -z .5 -p SearchAgent -a fn=bfs
```

Task 3: Uniform-Cost Search (UCS)

Goal: Implement Uniform-Cost Search in `uniformCostSearch` inside `search.py`.

Technical Requirements:

- Use a `PriorityQueue` from `util.py` ordered by accumulated path cost $g(n)$.
- Test your implementation across varying step costs (e.g., mazes with directional costs like `stayEastSearch`).
- Handle frontier updates: if a newly generated successor offers a cheaper path cost $g(n)$ to a state already on the fringe, update its priority in the queue.

```
python pacman.py -l mediumMaze -p SearchAgent -a fn=ucs
python pacman.py -l mediumDenselyMaze -p SearchAgent -a fn=ucs
python pacman.py -l stayEastSearch -p SearchAgent -a fn=ucs
```

Task 4: Greedy Best-First Search (GBFS)

Goal: Implement Greedy Best-First Search in `greedyBestFirstSearch` (alias `gbfs`) inside `search.py`.

Technical Requirements:

- Use a `PriorityQueue` ordered strictly by heuristic value $h(n)$.
- Integrate command-line heuristic parameter passing so that different heuristics (e.g., Manhattan distance, Euclidean distance) can be passed dynamically.

```
python pacman.py -l bigMaze -z .5 -p SearchAgent -a fn=gbfs,heuristic=manhattanHeuristic
```

Task 5: A* Search

Goal: Implement A* Search in `aStarSearch` inside `search.py`.

Technical Requirements:

- Use a `PriorityQueue` ordered by evaluation function $f(n) = g(n) + h(n)$, where $g(n)$ is accumulated path cost and $h(n)$ is estimated heuristic cost to goal.
- Verify optimality: ensure A* returns the true shortest cost path when paired with an admissible and consistent heuristic.

```
python pacman.py -l bigMaze -z .5 -p SearchAgent -a fn=astar,heuristic=nullHeuristic
python pacman.py -l bigMaze -z .5 -p SearchAgent -a fn=astar,heuristic=manhattanHeuristic
```

Task 6: Multi-Goal State Space (Corners Problem)

Goal: Formulate a new state space search problem in `searchAgents.py` to touch all four corners of a maze in minimal steps.

Technical Requirements:

- Define state representation in CornersProblem: a tuple containing (pacman_position, tuple_of_visited_corners).
- Implement cornersHeuristic in searchAgents.py. The heuristic must be mathematically admissible (never overestimate cost) and consistent.

```
python pacman.py -l tinyCorners -p SearchAgent -a fn=bfs,prob=CornersProblem
```

```
python pacman.py -l mediumCorners -p AStarCornersAgent -z .5
```

Task 7: Eating All Food Dots & Nearest Food Search

Goal: Design specialized search solutions for complex multi-dot food collection in searchAgents.py.

Technical Requirements:

- Implement foodHeuristic for FoodSearchProblem using minimum spanning trees (MST) or bottleneck distance calculations to optimize node expansions on trickySearch.
- Implement AnyFoodSearchProblem to allow Pacman to quickly locate and navigate to the single closest food dot using BFS.

```
python pacman.py -l trickySearch -p AStarFoodSearchAgent
```

```
python pacman.py -l bigSearch -p ClosestDotSearchAgent
```

3. Automated CSV Trace Logging Specification

To prove algorithmic correctness and generate automated trace data, every algorithm execution must write execution logs state-by-state to a CSV file inside the evidence/ folder.

Mandatory CSV Columns:

```
iteration, expanded_state, parent, action, generated_successors, frontier_before, frontier_after, explored, g, h, f
```

4. Custom Maze Creation & Experiments

Create an original layout file layouts/[YourID]Search.lay (e.g., 2111234Search.lay). The maze must contain multiple decision branches, dead ends, and deceptive traps designed to show differences between Greedy Best-First Search and A* Search. Test all 5 algorithms on your layout and include screenshot comparisons in your report.

5. Final Submission Package Requirements

Your final submission must be a single ZIP archive containing the full project folder with all original starter code, modified files, custom layout, written PDF report, and **ALL generated CSV trace logs inside the evidence/ directory.**

Example zip format:

```
[SearchProject.zip
|
|— search.py          (Edited: DFS, BFS, UCS, GBFS, A*, CSV Logger)
|— searchAgents.py    (Edited: CornersProblem, FoodHeuristic, etc.)
|— pacman.py         (Original starter file - DO NOT MODIFY)
|— game.py           (Original starter file - DO NOT MODIFY)
```

- |— util.py (Original starter file - DO NOT MODIFY)
- |— layout.py (Original starter file - DO NOT MODIFY)
- |— README.txt (Contains Python version, run commands, system specs)
- |— report.pdf (Detailed 6-10 page analysis report)
- |
- |— layouts/
 - |— *.lay (Standard starter layouts)
 - |— [YourID]Search.lay (Your custom maze layout file)
- |
- |— evidence/
 - |— *.csv (ALL complete execution CSV logs for all runs)
 - |— screenshots/ (Visual maze solution graphics)

Student Name	Roll number

Evaluation Rubric (Bring its print during viva)

Evaluation Category	Allocated Marks	Specific Assessment Criteria	Obtained Marks
Automated Test Cases Execution	20 Marks	100% passing rate on standard tests across all 5 algorithms and search problems.	
Live Demonstration / Viva Defense	20 Marks	Live code walkthrough, answering line-by-line questions, and running algorithms on unseen mazes.	
A* Search Implementation	10 Marks	Flawless $f(n)=g(n)+h(n)$ queue ordering, priority updates, and optimal path finding.	
Uniform-Cost Search (UCS) Implementation	10 Marks	Accurate path-cost accumulated tracking $g(n)$ and re-prioritizing frontier nodes.	
Heuristic Design & Analysis	10 Marks	Admissibility and consistency mathematical proofs, non-trivial custom heuristics.	
Depth-First Search (DFS) Implementation	10 Marks	Correct LIFO graph search, visited state set management, valid action sequences.	
Breadth-First Search (BFS) Implementation	10 Marks	Correct FIFO queue implementation, optimal path length guarantees on unweighted mazes.	
Greedy Best-First Search (GBFS) Implementation	10 Marks	Strict $h(n)$ heuristic prioritization and command-line integration.	
Multi-Goal Search Tasks	10 Marks	CornersProblem state tuple design and FoodSearch heuristics optimization.	
Automatic CSV Trace Logging	10 Marks	Full step-by-step CSV log files generated inside evidence/ directory for all tasks.	
Custom Maze & Experiments	10 Marks	Well-designed .lay custom maze, empirical performance tables, and screenshots.	

Report & Complexity Analysis	10 Marks	Comprehensive PDF report (6-10 pages) analyzing space/time complexity and state spaces.	
Code Quality & Style	10 Marks	Clean Python code formatting, comments, and strict adherence to provided interface rules.	
TOTAL MARKS	150 Marks		

REMARKS: